Pearls AQI Predictor

Project Report

Submitted By:

Areesha Fatima

1. Introduction

This project focuses on building an end-to-end Air Quality Index (AQI) prediction system using real-time data ingestion, feature engineering, machine learning models, a model registry, and a web application for inference and visualization. The goal was to train models and design a production-style ML pipeline.

2. System Architecture Overview

- **Data Source**: Weather & AQI APIs
- **Feature Store**: MongoDB collection (aqi_features)
- **Models**: Ridge Regression, SVR, Random Forest, Logistic Regression
- **Model Registry**: MongoDB collection (model_registry)
- **Frontend**: Streamlit for visualization

3. Data Collection & Feature Store

3.1 APIs Used

- AQI data - OpenWeather for Air Quality
  https://api.openweathermap.org/data/2.5/air_pollution
- Weather data (temperature, humidity, etc.) - OpenMeteoAPI for Weather
  https://api.open-meteo.com/v1/forecast

3.2 Data Ingestion Pipeline

The data ingestion pipeline performs the following steps:

1. Fetch raw AQI and weather data from APIs
2. Validate and clean missing or inconsistent values
3. Convert timestamps to a standardized format
4. Store raw and processed values for feature engineering

3.3 Feature Engineering

Each record stored in MongoDB contains:

- Temporal features: *hour, day, month, day_of_week, is_weekend*
- Pollution features: *pm2_5, pm10*
- Weather features: *temperature, humidity*
- Derived features: *aqi_change, aqi_3h_avg, aqi_12h_avg, pm_ratio*

These features are stored in the Feature Store (MongoDB) for reuse during training and inference.

4. Machine Learning Models

Four regression models were implemented:

4.1 Ridge Regression

- Linear model with L2 regularization
- Fast to train and interpretable

4.2 Support Vector Regression (SVR)

- Non-linear regression using RBF kernel
- Better performance on complex relationships

4.3 Random Forest Regressor

- Ensemble of decision trees
- Handles non-linearities and feature interactions well

4.4 Logistic Regression

- Used for AQI category prediction (e.g., Good, Moderate, Poor)
- Treats AQI prediction as a classification problem
- Useful for alert generation and threshold-based warnings

(However, after multiple training it was observed that logistic regression completely overfit the data therefore was not the appropriate choice for future training)

5. Training Pipeline

Each model follows a standardized training pipeline:

1. Load historical feature data from MongoDB Feature Store
2. Split data into training and testing sets
3. Scale numerical features using StandardScaler
4. Train the model
5. Evaluate performance using:
    - RMSE
    - MAE
    - R^2 Score
6. Store evaluation metrics in the Model Registry

6. Challenges Faced and Solutions

6.1 API Data Inconsistencies

- Issue: Missing or inaccurate weather values
- Solution: Switched APIs and added validation checks

6.2 Identifying Target Variable

- Issue: ‘aqi’ producing inaccurate predictions
- Solution: Used ‘pm_25’ as the target variable and used the formula:

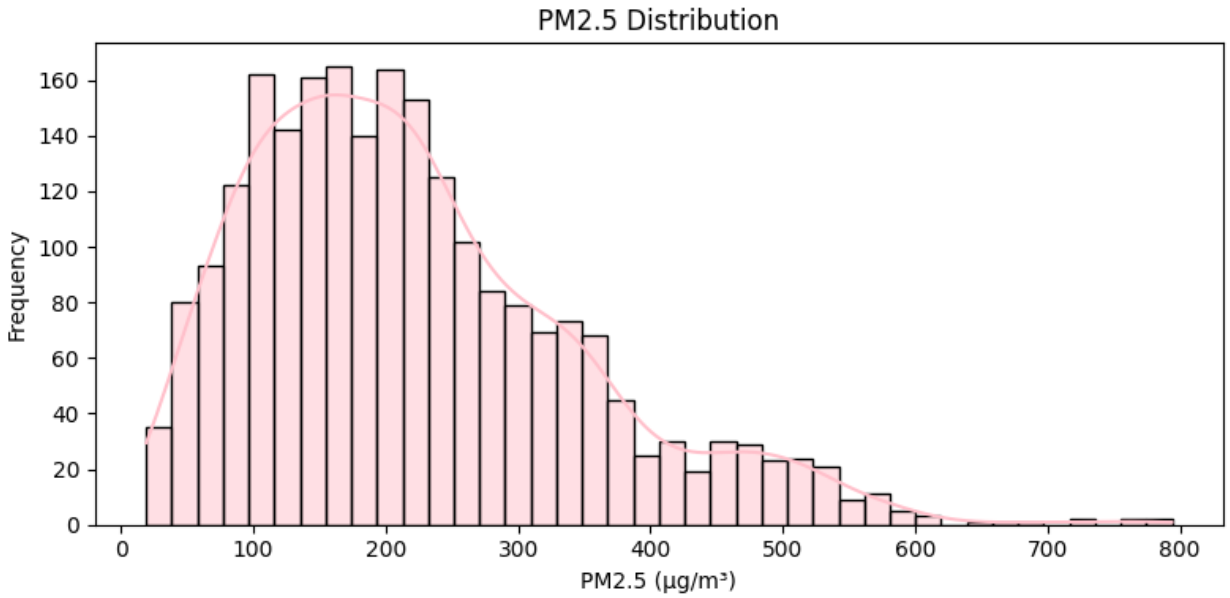
AQI = ((PM2.5 − lower_PM2.5_limit) / (upper_PM2.5_limit − lower_PM2.5_limit)) × (upper_AQI − lower_AQI) + lower_AQI

6.3 Understanding Overall Architecture

- Considering this was a first fully automated data science pipeline project, therefore I learnt quite a few new concepts and figured out on how to work with Hopswork, Mongodb, Streamlit as well as techniques on how things work.

7. Exploratory Data Analysis

7.1 Distribution Plot PM_2.5

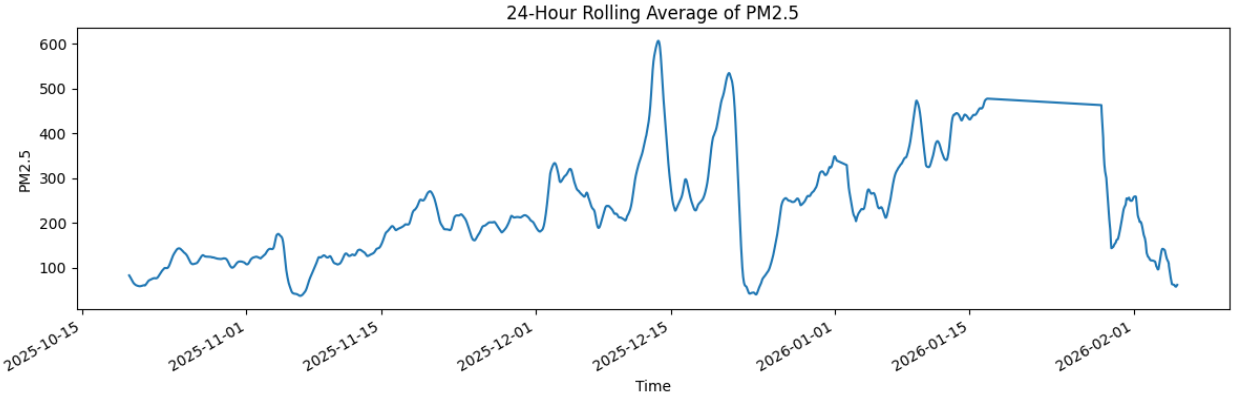


The distribution of PM2.5 values is right-skewed, indicating that lower to moderate pollution levels occur more frequently, while extreme pollution events are comparatively rare. This pattern is typical for air quality data, where occasional spikes correspond to environmental or human-induced events.

This helps:

- Highlights need to handle outliers
- Confirms real-world pollution behavior
- Guides model selection and evaluation

7.2 Time-Series Trend Plot



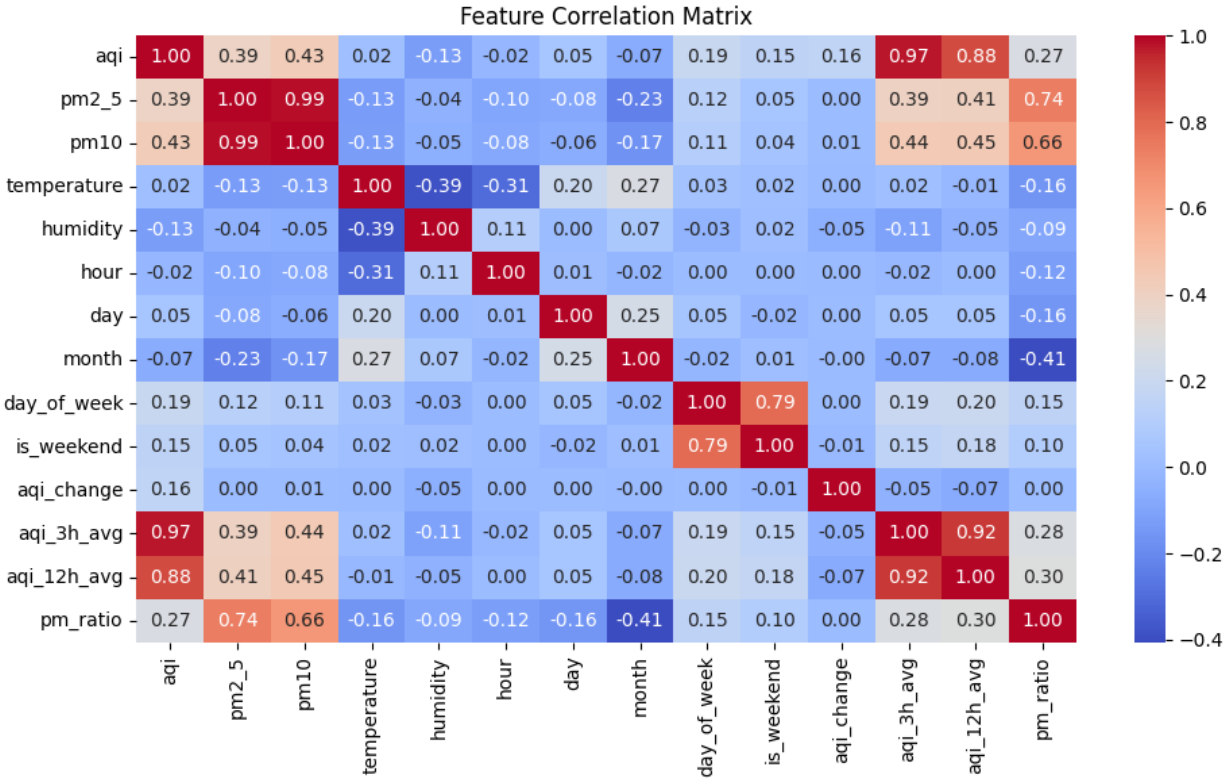
The time-series plot reveals clear temporal variation in PM2.5 concentrations, including short-term fluctuations and longer-term trends. This confirms that air quality exhibits temporal dependency, validating the use of time-aware modeling approaches.

Seasonal patterns and recurring peaks indicate that past observations contain predictive information about future air quality. This supports the feasibility of forecasting PM2.5 and hence AQI for the next three days using historical data.

The trend:

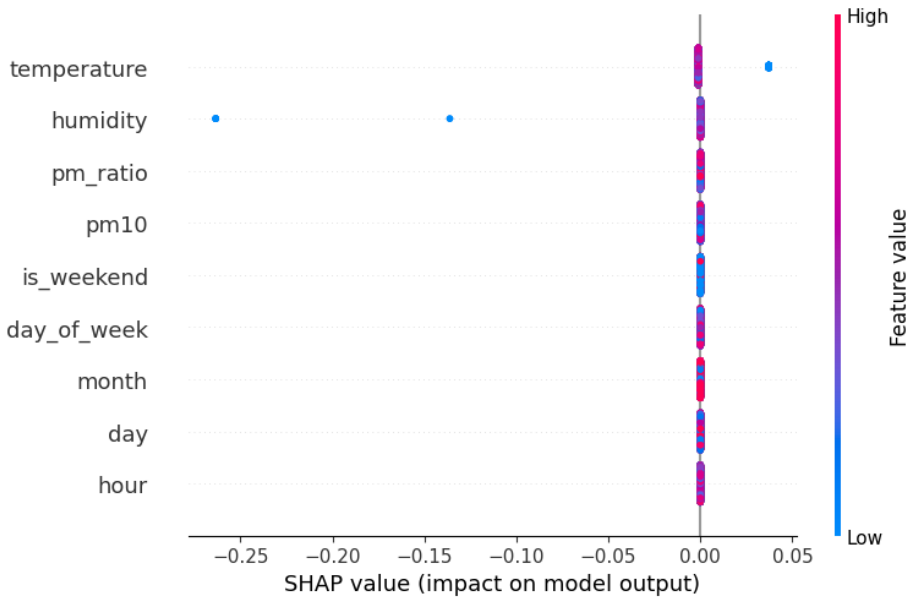
- Confirms forecasting is appropriate
- Demonstrates temporal dependency
- Supports time-series feature engineering (lags, rolling means)

7.3 Feature Correlation Matrix



The correlation matrix shows that PM2.5 has meaningful relationships with several meteorological variables such as temperature, humidity, wind speed, and atmospheric pressure. Some variables exhibit positive correlations indicating that PM2.5 increases with the feature, while others show negative correlations indicating a mitigating effect.

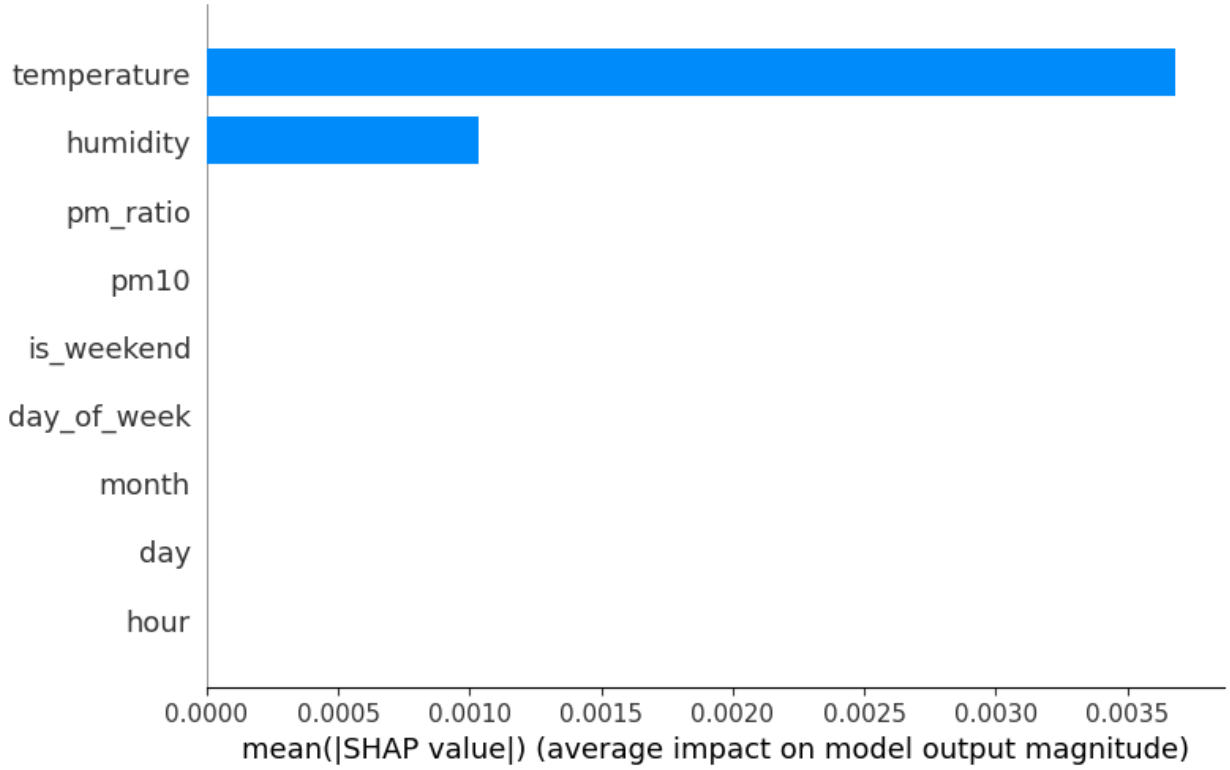
8. Feature Importance Explanations using SHAP



The SHAP summary plot illustrates the global importance of input features in predicting PM2.5 concentration. Features at the top of the plot exhibit the highest overall impact on model output, indicating that the model relies most heavily on these variables when forecasting air quality.

The color gradient reflects the feature value magnitude, showing how high and low values influence predictions. For example, higher values of certain meteorological variables correspond to increased predicted PM2.5 levels, while lower values contribute to reduced predictions. This demonstrates that the model captures meaningful environmental relationships rather than producing arbitrary predictions.

8.1 Mean Absolute SHAP



The bar plot presents the average magnitude of each feature's contribution to the model's predictions. Variables with higher mean absolute SHAP values have a stronger influence on PM2.5 forecasts.

The dominance of meteorological features such as temperature, humidity, wind speed, and pressure indicates that air quality predictions are primarily driven by physical atmospheric conditions rather than noise. This supports the inclusion of these features as core inputs in the predictive pipeline.

Therefore, both global and local explanations confirm that predictions are driven by a combination of temperature, humidity, wind speed, and pressure, among other factors. The ability to explain individual forecasts enhances transparency and trust, which is essential for deploying the system as an automated, serverless air quality prediction service.

Conclusion

This project implements a complete end-to-end system for predicting the Air Quality Index (AQI) in the next three days using a 100% serverless approach. The project covers the entire machine learning lifecycle, including data collection, feature engineering, model training, evaluation, storage, inference, and visualization. Raw environmental data is automatically fetched, transformed into useful features, and used to train and select the best-performing model, which is then deployed to generate future AQI forecasts. The system also includes model explainability using SHAP and an interactive dashboard to display real-time and predicted AQI values, along with alerts for hazardous conditions. Overall, this project demonstrates how an automated and scalable pipeline can be built to support reliable and practical air quality forecasting.